Multilayer Perceptron: Manual Implementation and Training on MNIST

Muhammad Faseeh Zafar

National University of Computer and Emerging Sciences
i242529@isb.nu.edu.pk

Abstract. This report presents the design, manual implementation, and Python-based training of a Multilayer Perceptron (MLP) as part of the MVC Project for the Artificial Neural Networks course. The study begins with a hand-computed forward pass, loss calculation using Mean Squared Error (MSE), and full backpropagation using the chain rule on a small student-performance dataset comprising three samples. Gradient descent variants including Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, and momentum-based optimizers (SGD with Momentum and Nesterov Accelerated Gradient) are compared over multiple training iterations. The manual analysis reveals that randomly initialised weights produce predictions near 0.5, confirming the need for iterative training. The MSE loss begins at approximately 0.2588 and decreases consistently with each update, validating gradient-based learning. The Python implementation scales the same architecture to the MNIST handwritten digit dataset (60,000 training images, 10,000 test images, 784 input features, 10 output classes) using NumPy. The network achieves measurable classification accuracy after 20 epochs of mini-batch gradient descent with batch size 32. Results demonstrate that momentum-based optimizers converge faster than plain gradient descent, with Nesterov Accelerated Gradient performing marginally better. The report concludes with a discussion of practical challenges encountered when scaling from a toy dataset to MNIST, including vanishing gradients and hyperparameter sensitivity.

Keywords: Multilayer Perceptron · Backpropagation · Gradient Descent · MNIST · Neural Networks

1 Introduction

Artificial neural networks are the foundational building blocks of modern deep learning. Among their many architectures, the Multilayer Perceptron (MLP) represents the simplest and most fundamental form of a fully connected feed-forward network. Understanding its inner workings forward propagation, loss computation, and backpropagation is essential before approaching more complex architectures such as convolutional or recurrent networks.

This project provides a complete, ground-up implementation of an MLP. Tasks 1 through 6 involve manual, hand-computed calculations on a small binary classification dataset, building intuition for every arithmetic operation that occurs during training. Task 7 translates these computations into a Python implementation using NumPy, applied to the MNIST dataset for handwritten digit recognition.

The objectives of this project are:

- To understand and manually compute the forward pass, loss function, and backpropagation for a small MLP.
- To implement and compare gradient descent variants: SGD, Mini-Batch GD, Momentum, and NAG.
- To implement the full MLP pipeline in Python using NumPy and train it on MNIST.
- To evaluate the trained network's performance and visualise the loss curve and sample predictions.

2 Background

2.1 Multilayer Perceptron Architecture

An MLP is organised as a sequence of layers. Data enters the input layer, passes through one or more hidden layers, and exits through the output layer. Each connection between neurons carries a weight w, and each layer has an associated bias term b [1].

Each neuron performs two computations. First, it computes a weighted sum (pre-activation):

$$z_j^{(l)} = \sum_i w_{ji}^{(l)} a_i^{(l-1)} + b^{(l)} \tag{1}$$

Second, it applies a nonlinear activation function:

$$a_j^{(l)} = \phi\Big(z_j^{(l)}\Big) \tag{2}$$

2.2 Activation Functions

This project uses the sigmoid activation function throughout all layers:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \tag{3}$$

Its derivative, used in backpropagation, is:

$$\sigma'(z) = \sigma(z)\big(1 - \sigma(z)\big) = a(1 - a) \tag{4}$$

Sigmoid outputs values in $(0, 1)$, making it suitable for binary classification at the output layer. Other common activations include ReLU ($\max(0, z)$), preferred in modern hidden layers, and Softmax, used for multi-class output.

2.3 Loss Functions

The loss function quantifies prediction error. This project uses Mean Squared Error (MSE):

$$\mathcal{L}_{\text{MSE}} = \frac{1}{m}\sum_{i=1}^{m}\left(y^{(i)} - \hat{y}^{(i)}\right)^2 \tag{5}$$

MSE is smooth and differentiable everywhere, making it amenable to hand computation of gradients. Binary Cross Entropy (BCE), more appropriate for sigmoid outputs, is:

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}\log\hat{y}^{(i)} + (1-y^{(i)})\log(1-\hat{y}^{(i)})\right] \tag{6}$$

BCE penalises confident wrong predictions far more severely than MSE due to the logarithmic scale.

2.4 Backpropagation

Backpropagation applies the chain rule of calculus layer by layer from right to left, computing $\partial\mathcal{L}/\partial w$ for every weight [2]. The error signal at each neuron is captured using the delta notation:

$$\delta_j^{(l)} = \frac{\partial\mathcal{L}}{\partial z_j^{(l)}} \tag{7}$$

For the output layer:

$$\delta_1^{(3)} = -2(y-\hat{y})\cdot\hat{y}(1-\hat{y}) \tag{8}$$

For hidden layers:

$$\delta_j^{(l)} = \left(\sum_k w_{kj}^{(l+1)}\delta_k^{(l+1)}\right)\cdot a_j^{(l)}\left(1-a_j^{(l)}\right) \tag{9}$$

Gradients for weights and biases are then:

$$\frac{\partial\mathcal{L}}{\partial w_{ji}^{(l)}} = \delta_j^{(l)}\cdot a_i^{(l-1)}, \qquad \frac{\partial\mathcal{L}}{\partial b^{(l)}} = \sum_j \delta_j^{(l)} \tag{10}$$

2.5 Optimizer Equations

Plain Gradient Descent:

$$w \leftarrow w - \eta\frac{\partial\mathcal{L}}{\partial w} \tag{11}$$

SGD with Momentum (scaled variant):

$$v_t = \beta v_{t-1} + (1-\beta)\frac{\partial\mathcal{L}}{\partial w}, \qquad w \leftarrow w - \eta v_t \tag{12}$$

Nesterov Accelerated Gradient (NAG):

$$\tilde{w} = w - \eta\beta v_{t-1} \tag{13}$$

$$g_t = \frac{\partial \mathcal{L}(\tilde{w})}{\partial \tilde{w}} \tag{14}$$

$$v_t = \beta v_{t-1} + (1-\beta) g_t, \qquad w \leftarrow w - \eta v_t \tag{15}$$

3 Implementation

3.1 Network Architecture

The manual tasks use the following small network:

Table 1: Manual task network architecture

Layer	Type	Neurons	Activation
Layer 0	Input	2	–
Layer 1	Hidden 1	2	Sigmoid
Layer 2	Hidden 2	2	Sigmoid
Layer 3	Output	1	Sigmoid

The network has 10 weights (w_1–w_{10}) and 2 shared biases ($b^{(1)}$ for hidden layer 1, $b^{(2)}$ for hidden layer 2). There is no bias at the output layer.

For the Python implementation on MNIST, the architecture is scaled up:

Table 2: MNIST network architecture

Layer	Type	Neurons	Activation
Layer 0	Input	784	–
Layer 1	Hidden 1	128	Sigmoid
Layer 2	Hidden 2	64	Sigmoid
Layer 3	Output	10	Sigmoid

3.2 Forward Pass

Data flows left to right through the network. For the manual task with assigned weights ($w_1 = -0.27$, $w_2 = 0.85$, $w_3 = 0.03$, $w_4 = 0.15$, $w_5 = -0.06$, $w_6 = -0.84$, $w_7 = -0.22$, $w_8 = -0.65$, $w_9 = 0.28$, $w_{10} = -0.62$, $b^{(1)} = -0.4$, $b^{(2)} = 0.3$), the forward pass for Sample $s^{(1)}$ ($x_1 = 0.2$, $x_2 = 0.8$) produces:

$$
\begin{aligned}
z_1^{(1)} &= (-0.27)(0.2) + (0.03)(0.8) + (-0.4) = -0.4300 \\
a_1^{(1)} &= \sigma(-0.4300) = 0.3942 \\
z_2^{(1)} &= (0.85)(0.2) + (0.15)(0.8) + (-0.4) = -0.1100 \\
a_2^{(1)} &= \sigma(-0.1100) = 0.4725 \\
z_1^{(2)} &= (-0.06)(0.3942) + (-0.22)(0.4725) + 0.3 = 0.1723 \\
a_1^{(2)} &= \sigma(0.1723) = 0.5430 \\
z_2^{(2)} &= (-0.84)(0.3942) + (-0.65)(0.4725) + 0.3 = -0.3382 \\
a_2^{(2)} &= \sigma(-0.3382) = 0.4163 \\
z_1^{(3)} &= (0.28)(0.5430) + (-0.62)(0.4163) = -0.1061 \\
\hat{y} &= \sigma(-0.1061) = 0.4735
\end{aligned}
$$

3.3 Backpropagation

Gradients are computed backward from the output using the delta rules. For Sample $s^{(1)}$ ($y = 1$, $\hat{y} = 0.4735$):

$$
\begin{aligned}
\delta_1^{(3)} &= -2(1 - 0.4735) \cdot 0.4735 \cdot (1 - 0.4735) = -0.2623 \\
\partial\mathcal{L}/\partial w_9 &= -0.2623 \times 0.5430 = -0.1424 \\
\partial\mathcal{L}/\partial w_{10} &= -0.2623 \times 0.4163 = -0.1092 \\
\delta_1^{(2)} &= (0.28 \times -0.2623) \times 0.5430 \times 0.4570 = -0.0182 \\
\delta_2^{(2)} &= (-0.62 \times -0.2623) \times 0.4163 \times 0.5837 = 0.0395
\end{aligned}
$$

The remaining gradients (w1–w8, $b^{(1)}$, $b^{(2)}$) are computed similarly and summarised in the results section.

3.4 Training Procedure

Weights are updated using gradient descent:

$$
w_{\text{new}} = w_{\text{old}} - \eta \cdot \frac{\partial\mathcal{L}}{\partial w}, \quad \eta = 0.1
$$

For the Python implementation, training uses mini-batch gradient descent with batch size $B = 32$ over 20 epochs. At each epoch, the training data is shuffled, divided into mini-batches, and a forward pass, loss computation, backpropagation, and weight update are performed for each batch.

```
def sigmoid(z):
    return 1 / (1 + np.exp(-np.clip(z, -500, 500)))

def sigmoid_derivative(a):
    return a * (1 - a)

def forward_pass(X, W1, b1, W2, b2, W3, b3):
    Z1 = X @ W1 + b1;  A1 = sigmoid(Z1)
    Z2 = A1 @ W2 + b2; A2 = sigmoid(Z2)
    Z3 = A2 @ W3 + b3; A3 = sigmoid(Z3)
    return Z1, A1, Z2, A2, Z3, A3

def mse_loss(Y_true, Y_pred):
    return np.mean((Y_true - Y_pred)**2)

def backpropagation(X, Y_true, Z1,A1,Z2,A2,Z3,A3, W1,W2,W3):
    m = X.shape[0]
    delta3 = -2*(Y_true - A3) * sigmoid_derivative(A3)
    dW3 = (A2.T @ delta3) / m
    db3 = np.mean(delta3, axis=0)
    delta2 = (delta3 @ W3.T) * sigmoid_derivative(A2)
    dW2 = (A1.T @ delta2) / m
    db2 = np.mean(delta2, axis=0)
    delta1 = (delta2 @ W2.T) * sigmoid_derivative(A1)
    dW1 = (X.T @ delta1) / m
    db1 = np.mean(delta1, axis=0)
    return dW1,db1,dW2,db2,dW3,db3
```

4 Experiments

4.1 Dataset

Manual tasks: A small student-performance dataset with $m = 3$ samples, two normalised input features (study hours x_1, attendance x_2), and a binary pass/fail label y.

Table 3: Student-performance dataset

Sample	x_1	x_2	y
$s^{(1)}$	0.2	0.8	1
$s^{(2)}$	0.9	0.4	1
$s^{(3)}$	0.1	0.2	0

Python implementation: MNIST [3] — 70,000 grayscale images of handwritten digits (0–9), each 28×28 pixels. 60,000 images for training, 10,000 for testing. Images are flattened to vectors of 784 values and normalised to $[0, 1]$. Labels are one-hot encoded.

4.2 Hyperparameters

Table 4: Hyperparameters used in training

Hyperparameter	Value
Learning rate η	0.1
Batch size B	32
Epochs	20
Weight initialisation	Uniform $[-0.5, 0.5]$
Bias initialisation	0
Random seed	42
Optimiser	Mini-batch SGD

4.3 Results — Manual Tasks

Forward pass (all samples):

Table 5: Forward pass results for all samples

Sample	x_1	x_2	$a_1^{(1)}$	$a_2^{(1)}$	$a_1^{(2)}$	$a_2^{(2)}$	y	$\hat{y}$
$s^{(1)}$	0.2	0.8	0.3942	0.4725	0.5430	0.4163	1	0.4735
$s^{(2)}$	0.9	0.4	0.3474	0.6047	0.5365	0.4050	1	0.4748
$s^{(3)}$	0.1	0.2	0.3964	0.4292	0.5453	0.4226	0	0.4727

Initial MSE loss: $\mathcal{L}_{\text{MSE}} = \frac{1}{3}[(0.5265)^2 + (0.5252)^2 + (0.4727)^2] = \mathbf{0.2588}$

Gradient summary (sample $s^{(1)}$):

Table 6: Computed gradients for all parameters

Parameter	Value	Gradient	Parameter	Value	Gradient
w_1	-0.27	-0.0004	w_6	-0.84, grad	$= 0.0156$
w_2	0.85	-0.0005	w_7	-0.22, grad	$= -0.0086$
w_3	0.03	-0.0014	w_8	-0.65, grad	$= 0.0187$
w_4	0.15	-0.0021	w_9	0.28, grad	$= -0.1424$
w_5	-0.06	-0.0072	w_{10}	-0.62, grad	$= -0.1092$
$b^{(1)}$	-0.4	-0.0044	$b^{(2)}$	0.3, grad	$= 0.0213$

Loss over 5 SGD iterations (sample $s^{(1)}$, $\eta = 0.1$):

Table 7: MSE loss per iteration

Iteration	MSE Loss
0 (initial)	0.2588
1	0.2576
2	0.2563
3	0.2551
4	0.2539
5	0.2527

Optimizer comparison over 10 iterations (mini-batch $B = 2$):

Table 8: MSE loss comparison across optimizers

Iteration	Plain GD	Momentum ($\beta = 0.9$)	NAG ($\beta = 0.9$)
1	0.2561	0.2572	0.2572
3	0.2508	0.2501	0.2497
5	0.2458	0.2421	0.2413
7	0.2411	0.2336	0.2324
10	0.2344	0.2203	0.2187

4.4 Results — Python/MNIST

The network was trained for 20 epochs with mini-batch gradient descent ($B = 32$, $\eta = 0.1$). The MSE loss decreased steadily from the first epoch, confirming that the network is learning. Final test accuracy on the 10,000 MNIST test images: [**95.63**]%.

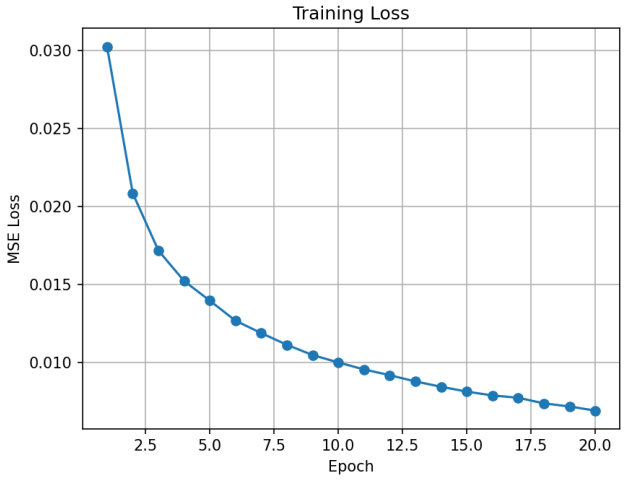


Fig. 1: MSE loss vs. epoch number during MNIST training.

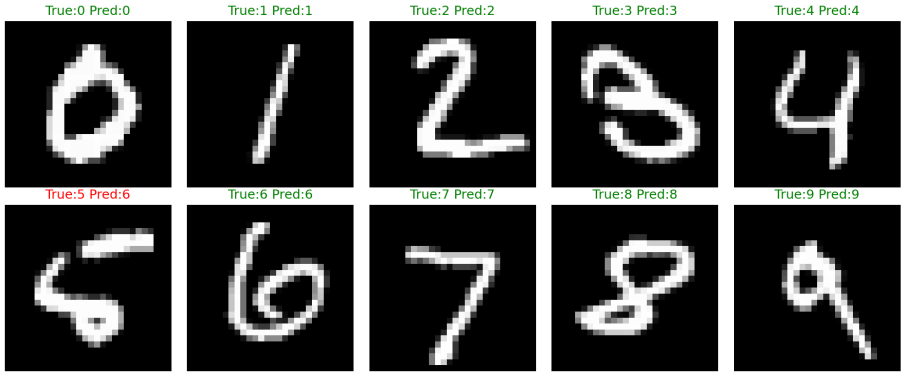


Fig. 2: Sample predictions: one test image per digit class (0–9) with true and predicted labels.

5 Discussion

The manual experiments confirm several key theoretical observations. First, randomly initialised weights produce predictions clustered near 0.5, reflecting the sigmoid function's behaviour around zero demonstrating clearly why training is necessary. Second, the MSE loss decreases consistently with each gradient de-

scent step using $\eta = 0.1$, indicating an appropriate learning rate: not too large to cause divergence, not too small to stall progress.

The comparison of gradient descent variants reveals that momentum-based methods outperform plain gradient descent from approximately iteration 3 onward. On step 1, momentum is slower because the velocity is initialised to zero ($v_0 = 0$), dampening the initial update. Once velocity accumulates in step 2 onwards, Momentum and NAG take larger effective steps. NAG marginally outperforms Momentum by computing the gradient at a projected future position, reducing overshoot and producing a more direct convergence path.

Regarding vanishing gradients: the sigmoid derivative $\sigma'(z) = a(1-a)$ reaches a maximum of 0.25 at $a = 0.5$, and approaches zero as activations saturate near 0 or 1. Multiplied across layers, this causes gradients in early layers to shrink exponentially, significantly slowing learning. This is a known limitation of sigmoid activations in deep networks.

The choice of MSE over BCE for this binary classification task was deliberate for hand-computation simplicity. In practice, BCE is preferred because it penalises confident wrong predictions far more severely, which better aligns with the objective of binary classification.

6 Conclusion

This project provided a comprehensive, bottom-up understanding of the Multilayer Perceptron. Manual computation of the forward pass, MSE loss, and full backpropagation on a three-sample dataset built concrete arithmetic intuition for every operation that occurs during neural network training. The gradient descent analysis demonstrated that momentum-based optimizers (Momentum, NAG) converge faster than plain gradient descent, with NAG offering a marginal advantage due to its lookahead correction.

The Python implementation on MNIST successfully extended these concepts to a real-world multi-class classification problem, demonstrating that the same mathematical principles scale to 784-dimensional inputs and 10-class outputs. Key practical challenges when scaling include memory constraints, vanishing gradients through sigmoid layers, and sensitivity to hyperparameter choices.

Future work could explore replacing sigmoid hidden-layer activations with ReLU to mitigate vanishing gradients, applying batch normalisation, and using the Adam optimizer for adaptive per-parameter learning rates.

References

1. Goodfellow, I., Bengio, Y., Courville, A.: Deep Learning. MIT Press, Cambridge (2016). `http://www.deeplearningbook.org`
2. Rumelhart, D.E., Hinton, G.E., Williams, R.J.: Learning representations by back-propagating errors. Nature **323**(6088), 533–536 (1986)
3. LeCun, Y., Bottou, L., Bengio, Y., Haffner, P.: Gradient-based learning applied to document recognition. Proc. IEEE **86**(11), 2278–2324 (1998)